MOVEINSYNC

Play Store Rating Analysis & Improvement Plan

Forward Deployed Engineer | Candidate Assignment

Prepared by

Debosmita Pal

Reg.No.- 12317734

Lovely Professional University

Email: debosmitapal429@gmail.com
Phone: 7583950759
Submission Date: 12/10/26

Objective

Identify the main drivers of negative employee-app reviews and propose a practical, measurable plan to improve reliability, customer experience, and Play Store ratings.

Data source: Public Google Play Store reviews collected using a JavaScript web-scraping workflow

1. Executive Summary

This report analyzes 1,500 Play Store reviews collected and organized in the supplied Excel workbook. The sample average rating is 3.81/5. 419 reviews (27.9%) are rated 1–2 stars, 39 (2.6%) are 3-star reviews, and 1,042 (69.5%) are rated 4–5 stars. The findings indicate that the rating problem should be addressed by identifying and fixing recurring failures in the employee commute journey—not by focusing on rating prompts alone.

Metric	Finding	Meaning
Reviews analyzed	1,500	Workbook sample; not necessarily all reviews available on Play Store.
Average sample rating	3.81/5	Baseline for this collected dataset only.
1–2 star reviews	419 (27.9%)	Primary pool for identifying friction and service failures.
4–5 star reviews	1,042 (69.5%)	Positive journeys should be protected while making fixes.
Developer replies	8 (0.5%)	Public reply coverage is low in the sample.
App version missing	150 (10.0%)	Version-level diagnosis is limited for these rows.

Decision in one sentence

Prioritize app stability and OTP/booking completion first, investigate live ETA and dispatch reliability next, and create a continuous review-monitoring loop that routes repeated complaint themes to the right owner.

2. Data and approach

The workbook covers reviews dated 16 Apr 2026 to 09 Oct 2026. The All Reviews sheet is the primary dataset; Executive Summary and Categorized Complaints were used as supporting workbook views. The analysis focuses on star distribution, recurring category labels, category-level ratings, dates, app-version availability, and developer-reply coverage.

- Collection: JavaScript-based web scraping of publicly visible Google Play Store review information.
- Preparation: exported the collected review records into Excel and organized the results into review, summary, and complaint-category sheets.
- Analysis: used Excel/workbook summaries to compare ratings and recurring complaint categories; recommendations are hypotheses to validate against product telemetry and trip operations.
- Important distinction: the workbook's mean rating is the average of this sample, not a verified current Play Store headline rating.

3. Key Findings from the Reviews

Category counts help show where to investigate first. The category is a classification of review text; it is not proof of a technical root cause.

Complaint category	Review count	Avg. rating	1–2★ reviews
General Dissatisfaction	200	1.08	200
OTP & Login / Auth Issues	53	1.60	44
App Crashes, Bugs & Performance	55	1.87	39
Live Tracking & GPS Navigation	69	2.71	37
UI / UX & Update Dissatisfaction	71	3.08	32
Cab Delay & Punctuality	29	1.97	22
Driver Behavior & Safety	50	3.28	19
Cab Allocation & Cancellation	13	1.23	12
Customer Support & Helpdesk	13	1.62	11
Battery Drain & Permissions	3	1.33	3
General Feedback	12	3.00	0

What these patterns suggest

- App stability and authentication deserve early attention because failures in these areas can block employees from using the app at a time-sensitive point in the commute.
- Live tracking, ETA, and punctuality complaints may involve several systems: GPS freshness, ETA calculation, dispatch, driver arrival, or the way status is presented in the app.
- UX and update-related complaints should be split into reproducible defects, confusing flows, and preference-related feedback instead of being handled as one broad category.
- Very low public developer-response coverage in the dataset suggests an opportunity to acknowledge actionable feedback and show users that issues are being investigated.
- The review sample should be paired with operational data. A review saying 'cab is late' does not alone tell us whether the root cause is app display, route planning, supply, traffic, or driver behavior.

Root-cause validation checklist

Signal in reviews	Evidence to check before assigning cause
Crashes / bugs	Crash-free sessions, ANRs, logs, OS/device mix, release version, rollout timing.
OTP / login	OTP delivery latency, retry rate, authentication errors, session expiry, device/network split.
Booking failure	Booking funnel drop-off, API errors, allocation state, duplicate requests, cancellation/rebooking events.
GPS / ETA / delays	GPS timestamp freshness, predicted vs actual pickup, route deviation, dispatch timestamps, time-of-day effects.
UX / updates	Usability sessions, event funnels, release notes, support contacts and controlled rollout comparisons.

4. Recommended Product and Engineering Actions

P0 — Fix blockers in core journeys

- App stability: reproduce high-frequency crash reports; inspect crash/ANR data by app version, OS and device; use staged rollouts and rollback criteria for risky releases.
- OTP and login: measure OTP delivery time and retry failures; improve clear error states and safe retry behavior; test session expiry and login across supported devices.
- Booking-state clarity: show whether a ride is requested, confirmed, assigned, failed, or cancelled. Avoid ambiguous states and give the employee a clear next action.

P1 — Make the commute more predictable

- Live tracking and ETA: show the last-updated time and a credible pickup estimate; identify stale GPS and compare predicted ETA with actual arrival by route and time band.
- Dispatch and punctuality: connect review themes to planned/actual pickup, route, allocation and driver-arrival records. Separate operational causes from app display issues.
- Safety issues: route any safety-related review through a high-priority, human-led escalation process. Do not leave safety complaints in an automated sentiment queue.

P2 — Improve experience and feedback handling

- Rating prompt: request a rating only after a completed journey, make it dismissible, and avoid repeated interruptions. Never gate functionality or pressure users to leave positive ratings.
- Review response: use issue-specific, empathetic replies and send reports into the same internal issue-tracking workflow used by support and engineering.
- Regression protection: add test cases for the top recurring complaints and track whether each release increases or decreases those complaint rates.

30-day execution plan

Timeline	Actions	Owners	Proof of progress
Days 1–3	Validate current public rating; review recent 1–2★ comments; create severity-ranked issue backlog.	FDE, Product, Support	Each issue has examples, owner, severity and next step.
Week 1	Instrument crash, OTP, booking and ETA journeys; segment by version/device; establish baselines.	Mobile, Backend, Data	Events and dashboards verified.
Weeks 2–3	Release low-risk fixes behind staged rollout; test booking clarity and OTP improvements.	Engineering, QA	Improved completion/stability with no safety regression.
Weeks 3–4	Investigate ETA/dispatch themes; pilot operational fixes; launch review triage workflow.	Routing/Ops, Product, FDE	Lower ETA error and fewer repeated complaint themes.
Day 30	Compare to baseline and decide the next set of fixes based on evidence.	Product, Data, FDE	A scorecard and ranked next-month backlog.

5. Final Recommendation: Build a Review Intelligence & Alerting Tool

The most scalable recommendation is a lightweight monitoring tool that continuously collects new public reviews, detects negative feedback, groups it into complaint categories, and alerts the responsible team when a theme spikes. This turns the assignment's recurring question—'How are our Play Store ratings?'—into an ongoing operational workflow rather than a one-time spreadsheet exercise.

Proposed workflow

Step	What the tool does	Output
1. Collect	Run a scheduled JavaScript scraper or approved data ingestion job to fetch new reviews, ratings, timestamps, app version and review IDs where available.	New reviews stored with source and collection time.
2. Detect	Flag 1–2★ reviews and significant rating drops; deduplicate reviews and normalize text.	Negative-review queue.
3. Classify	Use keyword rules or an NLP/LLM classifier to assign themes such as crash, OTP, booking, GPS/ETA, delay, UX, safety, or other.	Category, confidence score, sentiment and evidence phrase.
4. Detect spikes	Compare category counts and negative-review rates against a rolling baseline; use volume and rate thresholds rather than raw counts alone.	Spike alert with trend and example reviews.
5. Investigate	Attach app version, date, device/OS (if available), relevant logs and operational metrics; ask an owner to confirm root cause.	Triage ticket with evidence and severity.
6. Route & track	Create or update a ticket in the team's issue tracker; notify the owning team; track acknowledgement, fix, release and recurrence.	Accountable owner and resolution status.
7. Verify	After a fix, compare complaint rate, related product metrics and rating distribution before and after release.	Impact report and regression signal.

Alert logic: example starting point

- Immediate high-priority alert: a credible safety report, or a cluster of severe crash/login/booking failures. Safety reports must be reviewed by a human immediately under the company's established escalation policy.
- Category spike alert: alert when a category's negative-review count or share exceeds a configurable threshold versus its rolling 7- or 28-day baseline, with a minimum-volume guardrail.
- Version-specific alert: flag when a complaint theme rises sharply after a new app version or rollout.
- Noisy-data guardrails: deduplicate, exclude spam where possible, display classifier confidence, and let an analyst correct categories.

Dashboard essentials

- Current public Play Store rating and rating-count trend, clearly separated from the average rating of the scraped sample.
- New reviews, 1–2★ share, sentiment and complaint-category trends over time.
- Category by app version, dates and device/OS where available.
- Top example reviews, classifier confidence, ticket owner, severity, status, time to acknowledgement and time to resolution.
- Before/after view for each release or fix, including guardrail metrics such as crashes, booking completion and ETA accuracy.

6. Implementation, Governance and Success Metrics

Suggested MVP architecture

Layer	Practical implementation
Collection	Scheduled JavaScript/Node.js job using a permitted, reliable review source; retry logic, rate limits and collection-health alerts.
Storage	A small database or structured store keyed by review ID, with timestamps and ingestion metadata; retain only necessary public review data.
Analysis	Rule-based baseline first, then a text classifier; keep category definitions, confidence and human corrections.
Alerting	Email/Slack or the team's existing incident/ticketing channel; route by category and severity.
Dashboard	Simple dashboard with rating trend, complaint trends, app-version filters and open-ticket status.
Governance	Respect Google Play terms and applicable privacy rules; protect credentials; limit access; avoid collecting unnecessary personal data.

How to measure success

Metric	Why it matters	Cadence
Public Play Store rating	External outcome; confirm from Play Console or current listing.	Weekly
1–2★ share and category rate	Shows whether negative experience themes are reducing.	Weekly
Crash-free sessions / ANR	Confirms app stability changes are effective.	Daily
OTP and booking completion	Measures whether employees can complete critical steps.	Daily
ETA error and on-time pickup	Connects app feedback to real commute reliability.	Daily/weekly
Time to alert / acknowledge / resolve	Measures whether monitoring creates faster action.	Weekly
Repeat complaints after fix	Checks whether the root cause was actually addressed.	Per release

Risks and safeguards

- A review spike is an investigation signal, not automatic proof of a defect. Require evidence and owner confirmation before declaring root cause.
- Avoid auto-creating a flood of duplicate tickets: group similar reviews, deduplicate, apply minimum-volume thresholds, and support manual triage.
- Do not let automated classification close safety issues or deploy code. Automation should detect, summarize, route and recommend; people own decisions and fixes.
- Scraping can break or be restricted by platform changes and terms. Monitor ingestion health and use an approved source/API where available.

Methodology and tools used

Reviews were collected from publicly visible Google Play Store review information using a JavaScript web-scraping workflow, then exported into Excel for organization and analysis. The workbook's All Reviews, Executive Summary and Categorized Complaints sheets were used to review rating distribution, dates, categories, category-level average ratings, app-version completeness and developer-response coverage. Then structured the report and convert the observed themes into an actionable plan. The collection workflow was JavaScript-based.

Conclusion

Fix the core commute blockers first, validate ETA and dispatch complaints against operational data, and make feedback triage continuous. A Review Intelligence & Alerting Tool is the long-term recommendation: it should detect category spikes, surface evidence, notify the right owners, track fixes, and verify whether complaints decline after release. That creates a repeatable loop from public feedback to product action and measurable customer impact.

GitHub Link: https://github.com/DebosmitaPal/Moveinsync_Assignment

Excel sheet Link:
https://docs.google.com/spreadsheets/d/1mXgcxw2y8u4VezHjTptl54x97_T5_4fp/edit?usp=sharing&ouid=115153166232207113766&rtpof=true&sd=true